GRAFT: Grid-Aligned FP4 Trellis Quantization for FP4-Native Language Models

Sub-4-bit storage of MXFP4 checkpoints that decodes to MXFP4 blocks

Emi Huang

Abstract

Frontier mixture-of-experts (MoE) models increasingly release their routed experts only in MXFP4 (4.25 bits per weight). Current methods that go below this rate treat the experts as continuous weights; EXL3, for example, rotates them with random Hadamard transforms and codes them with a Gaussian-matched trellis. An MXFP4 matrix, however, is a discrete source on a known grid, and at 3 bits its memoryless distortion–rate function is about 0.6× that of a Gaussian source of the same variance. We present GRAFT (GRid-Aligned FP4 Trellis), a 3-bit trellis code that keeps EXL3's trellis and state hash, drops the rotation and uses a grid-aligned value map, so that it decodes to MXFP4 blocks with a per-column output scale and a sparse tail correction. On the routed experts of DeepSeek-V4.1-Flash at 3.006 bits per weight, GRAFT's held-out expert error is 0.63× that of EXL3 at 3 bits (lower on all but 11 of the 15,358 experts with held-out rows) and 0.66× that of EXL3 refitted with GRAFT's own fitting objective. Its excess NLL over the native model is 0.0085 nats, against 0.0131 for EXL3, and 0.0095 when its MXFP4 blocks run through the model's own MXFP8×MXFP4 expert kernel. The smaller representation reduces weight traffic, but whether it also raises throughput depends on overlapping the decode with memory access, which we do not measure here.

1 Introduction

DeepSeek-V4-Flash and V4.1-Flash store every routed expert in MXFP4 [6, 33], which encodes the weights as 4-bit E2M1 codes $\{0, \pm 0.5, \pm 1, \pm 1.5, \pm 2, \pm 3, \pm 4, \pm 6\}$ with one power-of-two scale per 32 inputs, or 4.25 bits per weight. Because the experts hold almost all of the parameters, serving these models in less memory requires going below the native format. For most of these models no higher-precision checkpoint is published, and since the experts are trained with MXFP4 in the loop, such a checkpoint may not exist at all [6, 19, 40]. Current practice nonetheless treats the MXFP4 weights as continuous. EXL3 [37], a streamlined implementation of QTIP [36], dequantizes the experts, rotates them with random Hadamard transforms until they look i.i.d. Gaussian, and codes them with a Gaussian-matched trellis. GGUF builds below 4 bits instead re-round the dequantized weights onto llama.cpp's block grids.

Rotation is an appropriate prior for BF16 weights but not for MXFP4 weights. Every nonzero MXFP4 weight in a column is an E2M1 value times a power of two, and because E2M1 is nearly closed under doubling and halving, the weights of a column lie on a small union of grids. Such a source has finite entropy (3.7–3.9 bits per weight on DeepSeek-V4.1-Flash), and its memoryless distortion–rate function lies far below that of a Gaussian source, the curve that Gaussian-matched coders are designed for. Rotation preserves the grid structure but moves it into the joint distribution of each 128-coordinate Hadamard block, where a coder that models the rotated coordinates as i.i.d. Gaussian does not exploit it.

GRAFT keeps the parts of EXL3 that do not depend on rotation (the 16-bit bitshift trellis, the tile layout, the hashed state-to-value map and the error-feedback encoder) but decodes to MXFP4 blocks. It drops the rotation and maps EXL3's hash through one fused multiply-add and the GPU's E2M1 conversion, so that every 32-input block of a column decodes to E2M1 values times one power of two. Each block carries a one-bit flag, which is the block's MX scale and halves the block where the source sits an octave lower. A small band of states at the hash centre decodes to a power-of-two ladder for outlier weights, and the decoder applies these values as an exact sparse correction. Columns are scaled by a power of two and a fitted fp16 factor, and every matrix is coded at the same rate of 3 bits per weight.

Contributions.

1. **Grid-aware sub-native compression.** To our knowledge, GRAFT is the first method that compresses FP4-native checkpoints below 4 bits per weight by exploiting their grid. Prior work on these checkpoints stays at about 4 bits and treats the weights as continuous [14, 26].
2. **A discrete-source rate–distortion analysis** of block-scaled FP4 weights. At 3 bits the memoryless distortion–rate function of DeepSeek's weights is about

0.6× that of a Gaussian source, for which Gaussian-matched rotation-based coders are designed. The source is close to memoryless in our measurements, so we use the i.i.d. function as a reference (Section 3).

3. **A trellis code that decodes to the source's own format.** Grid-aligned trellis quantizers exist for uniform integer grids [20, 29, 42] and for FP4 values [32]. GRAFT decodes to OCP MXFP4 blocks on the grid of the checkpoint it compresses, with power-of-two block scales, a per-column output scale and a sparse correction. The block-scaled product therefore runs on existing MX tensor-core GEMMs, including the reference model's own expert kernel, with only the output scale and the sparse correction added (Section 4).
4. **Results on DeepSeek-V4.1-Flash** for all 15,360 routed experts, compared with EXL3 at 3 bits at the expert level and at the model level (excess NLL, KL and top-1 agreement on held-out text). Ablations separate the code from the fitting objective by refitting EXL3 with GRAFT's objective, and they compare block flags with a per-matrix octave codebook, a flat rate with a per-matrix rate allocation, and the trellis with 3-bit scalar codebooks (Section 5).

2 Background

MXFP4 MoEs. The OCP microscaling formats [33] store 32 consecutive values as low-bit elements that share one E8M0 (power-of-two) scale, and MXFP4 uses E2M1 elements. DeepSeek-V4.1-Flash has 40 MoE layers, each with 384 routed experts (top-6 routing, hidden size 5,120, expert intermediate size 2,304). Only the routed experts are stored in MXFP4, and only they are requantized in this work.

Trellis-coded quantization. TCQ [29] decodes reproduction values through a finite-state machine. QTIP [36] combines a *bitshift* trellis, whose state is the last L bits of the stream, with a *computed* codebook. EXL3 [37] uses $L = 16$ and codes each 16×16 tile, in tensor-core order, as one tail-biting ring of 256 steps. Its "MUL1" codebook is the byte sum $h(s) = \sum_b \text{byte}_b(s \cdot \texttt{0x83DCD12D} \bmod 2^{32})$ of the state s, an approximately Gaussian integer in $[0, 1020]$ with mean 510. EXL3 first rotates the weights with random Hadamard transforms [4, 35] and then rounds them as in GPTQ/LDLQ [4, 12], quantizing the rows in sequence with error feedback through the LDL factors of $H = \mathbb{E}[xx^\top]$ and a Viterbi search per 16-row block.

3 The MXFP4 source is discrete

We normalize each column j of an MXFP4 matrix by a power of two, $\tilde{w}_{ij} = w_{ij}/2^{p_j}$. Each normalized weight is still an E2M1 value times a power of two, and pooled over columns the values form a discrete source with small support. We compute its memoryless distortion–rate function under squared error with Blahut–Arimoto [1, 3] on the pooled values of all weights in 66 matrices (two experts in each of 11 layers). The reconstruction points are free. Starting from the source values and their midpoints, each point moves after every Blahut–Arimoto step to the centroid of the source values mapped to it, and each slope is iterated until the Lagrangian changes by less than 10^{-13}. Fixing the points at the source values and midpoints raises the distortion at 3 bits by only 0.9%. Every computed point is achievable, so the lower envelope of the two computations is an upper estimate of the memoryless distortion–rate function. We compare this estimate with $D_G(R) = \sigma^2 2^{-2R}$, the distortion–rate function of a Gaussian source of the same variance. Among sources of variance σ^2, the Gaussian has the largest $D(R)$ [5], whereas a discrete source with entropy H has $D(R) = 0$ for $R \geq H$. A random rotation leaves the vector source's $D(R)$ unchanged, but it moves the discreteness into the joint distribution of each 128-dimensional block. Coders such as EXL3 model the rotated coordinates as i.i.d. Gaussian, and their codebooks are designed for $D_G(R)$. EXL3's trellis comes within about 11% of $D_G(R)$ on Gaussian tiles, and on DeepSeek's own weights after EXL3's rotation its error stays at $1.11\times D_G(R)$ (Figure 1a). We do not prove that no rotation-based coder can do better, but GRAFT's codebook is built for the discrete source, which these coders do not model.

For a synthetic MXFP4 source with entropy 4.13 bits, the ratio of its distortion–rate function to the Gaussian one (reconstruction on the source values and their midpoints) is 0.866, 0.774, 0.602 and 0.338 at 2, 2.5, 3 and 3.5 bits. For DeepSeek-V4.1-Flash's weights in GRAFT's power-of-two column domain (entropy 3.73 bits), the ratio is 0.586 at 3 bits and 0.13 at 3.5. Three consequences follow for GRAFT. (i) The gain grows as the rate approaches the entropy, so 2.5–3.5 bits is the useful band for an MXFP4 source, and rate is worth more than it is for a Gaussian source. (ii) Scales must be commensurate with the grid. A non-power-of-two column scale moves the column off the shared grid, and on DeepSeek such free scales leave 9% more error than powers of two (Section 4). (iii) The optimal reproduction distribution puts its mass on the modal grid points and about 13% near zero, so an E2M1 alphabet suffices (Figure 3).

The source is also close to memoryless in our measurements, which motivates the i.i.d. reference. On twelve DeepSeek matrices the codes carry 3.88–3.90 bits each. Conditioning on the previous code along the input dimension lowers this by 0.02–0.03 bits, and conditioning on the previous two codes by 0.03–0.05. The neighbouring column's code does not lower it at all, and the block's scale lowers it by at most 0.14, a dependence that per-column scales and per-block flags capture. We also found no useful shared

linear subspace across experts, since the leading directions of half of a layer's experts capture only the isotropic share of the other half's weight energy (21% against 20% at rank 1,024 of 5,120).

4 GRAFT

GRAFT is a deterministic post-training quantizer for weights released on an MXFP4 grid. It codes each matrix $W \in \mathbb{R}^{k\times n}$ (k inputs, n outputs) at 3 bits per weight with a trellis whose decoded values are MXFP4 blocks, combined with per-column scales (Figure 2).

Trellis. GRAFT uses EXL3's 16-bit bitshift trellis, which codes each 16×16 tile in tensor-core order as one tail-biting ring of 256 steps. At 3 bits per weight the decoder shifts in three bits per step, so a tile holds 768 bits.

The grid-aligned codebook. GRAFT keeps EXL3's hash $h(s)$ but replaces the value map. A weight in row i and column j belongs to the MX block of rows $32\lfloor i/32\rfloor$ to $32\lfloor i/32\rfloor+31$ of column j. If this block has flag $f \in \{0,1\}$ ($f = 0$ in a matrix without flags), the weight's trellis state s decodes to

$$v(s) = 2^{-f}\begin{cases}\pm 2^m\{1, 1.5\} & |h-510| \le d,\\ 2\,\mathrm{E2M1}\left(\frac{h-510}{w_f}\right) & \text{otherwise,}\end{cases}$$

with $h = h(s)$. The bulk value is computed as $\mathrm{E2M1}_{\mathrm{RNE}}(\mathrm{fp32}((2^{23}+h)q_a^{(f)} + q_b^{(f)}))$ with $q_a^{(f)} = 1/w_f$ and $q_b^{(f)} = -(2^{23}+510)q_a^{(f)}$, which takes one fused multiply-add and the GPU's E2M1 conversion. In an unflagged block the bulk takes the 15 values $\{0, \pm1, \pm2, \pm3, \pm4, \pm6, \pm8, \pm12\}$, and a flagged block decodes to half these values, $\{0, \pm0.5, \ldots, \pm6\}$, with its own width w_1. A column normalized by one power of two contains blocks whose MXFP4 scale is an octave below the column's, and the flag gives these blocks the grid they need. The width w_f sets how the Gaussian-like hash spreads over the grid. A tail state takes its sign from bit 0 of s and its magnitude from $\rho = (s \gg 1) \bmod 26$ as $2^{3+\lfloor\rho/2\rfloor}$, multiplied by 1.5 if ρ is odd, which gives 26 magnitudes from 8 to 49,152 (Figure 3). Because the bulk map outputs zero at the hash centre, the band trades a small part of the zero mass for range. It covers 175 states (0.27%) at $d = 0$ and 6,991 (10.7%) at $d = 20$. The fit uses $d = 0$ on 99.4% of the matrices and never more than $d = 14$, and 71% of the others lie in the last three layers, where the outliers are.

Decoded representation. Outside the tail, $v = 2^{1-f}e$ with e an E2M1 value, so every (32-input block, column) pair of a GRAFT matrix decodes to an OCP MXFP4 block, that is, to E2M1 codes with the power-of-two block scale 2^{1-f}. The flag is the block's MX scale stored in one bit. The full reconstruction is $\widehat{W} = (B+T)\,\mathrm{diag}(c)$, where B holds these MXFP4 blocks, T is the sparse tail and $c_j = 2^{p_j} m_g \alpha_j$ is the column scale. The matrix B is MXFP4, but $\widehat{W}$ in general is not, because α_j is a free fp16 factor and tail values lie outside their block's range. Both are applied outside the block-scaled product. A tail value is an exact FP8 number, which a decoder applies as a sparse correction next to the block at the cost of one product per tail weight (0.03–0.07% of the weights in our fits). Every decoded value is 0 or $\pm 2^e\{1, 1.5\}$, an exact FP8 E5M2 number.

Scales. Each column is scaled by $2^{p_j} m_g \alpha_j$. The first factor is a power of two that brings to 3 the RMS of the column's weights that lie within 16× its RMS, so that a few huge weights, which the tail codes, do not inflate it. The second is a power-of-two multiplier $m_g \in \{1, 2, 4, 8\}$ for densely heavy columns, shared by each group of 16 columns (one tile wide). The third is an fp16 factor α_j, computed in closed form after the fit as the H-weighted least-squares fit of the decoded column. Free (non-power-of-two) scales give 9% more error.

Parameter selection. The widths, the tail band d, the group multipliers and the flags are all chosen with one quantity, the *fit-weighted tile error*

$$E = \sum_t \frac{1}{p_t} \sum_{(i,j)\in t} \delta_i s_j\,(\hat{w}_{ij} - w_{ij})^2,$$

which is the error of the trellis when it codes a 16×16 tile t on its own (without error feedback) under the per-weight costs of the fit (δ_i is the block-LDL diagonal of the shrunk H and s_j the column weight), summed over 256 tiles drawn with probability p_t proportional to their share of $\sum \delta_i s_j w_{ij}^2$. Without flags, (w_0, d) is chosen by coordinate descent, and each group's multiplier by the same error over all of the group's tiles. At those scales we try three flag rules, each with its own widths. The first flags every block for which rounding its scaled weights onto the halved grid gives the lower fit-weighted error, and the other two flag the blocks whose largest scaled magnitude is at most 4 or at most 5. A rule with flags is charged a factor of $4^{1/32}$, which is what its $1/32$ bit per weight is worth on the Gaussian rate curve, and the cheapest of the four options is fitted. Sampling the tiles by importance under the fit's costs matters, because uniform, unweighted tiles misjudge matrices whose error concentrates in a few heavy rows.

Objective and encoder. For each matrix the fit minimizes $\sum_j s_j (w_j - \hat{w}_j)^\top H (w_j - \hat{w}_j)$ over its columns w_j. In the input-by-output convention, a gated MLP with gate pre-activations $z = W_1^\top x$ and up activations $u = W_3^\top x$ computes $y = W_2^\top\left(\phi(W_1^\top x)\odot W_3^\top x\right)$ (with the model's activation

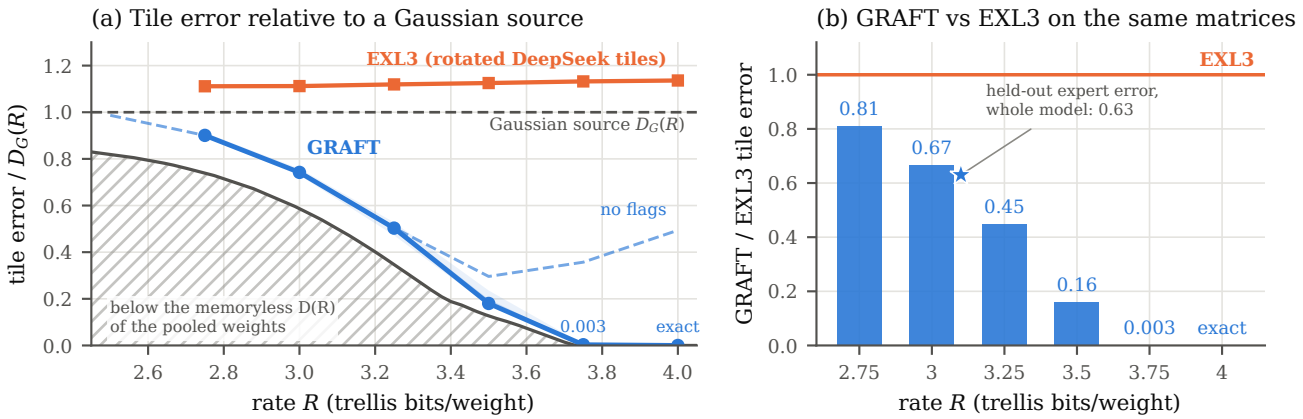


Figure 1: GRAFT's tile error relative to a Gaussian source. (a) Raw-tile error divided by $D_G(R) = 2^{-2R}$, the distortion–rate function of a Gaussian source of the same variance. GRAFT (solid; block flags where they pay for their bit, per-matrix widths, tail band 0) on every routed-expert matrix of 11 layers of DeepSeek-V4.1-Flash (12,672 matrices, 96 seeded tiles each; median, interquartile and 10–90% bands) reaches 0.74× at 3 bits, 0.18× at 3.5 and 0.003× at 3.75, and at 4 bits it reproduces every sampled tile of 91% of the matrices exactly. Dashed: the same codebook without flags on all 46,080 matrices. It stops improving above 3.5 bits (0.49× at 4), because blocks whose MX scale is an octave below their column's need the halved grid, which only the flags provide. Squares: EXL3's codebook on 66 of these matrices after EXL3's rotation (32 tiles each, median), at about 1.11× $D_G(R)$; on Gaussian tiles it comes within 11% of $D_G(R)$. Hatched: below the memoryless distortion–rate function of the pooled weights (entropy 3.73 bits), a reference for the per-matrix medians rather than a bound for each matrix. The rate counts trellis bits only. (b) GRAFT's tile error relative to EXL3's on the same 66 matrices (median of the per-matrix ratios); star: GRAFT's held-out expert error relative to EXL3 over the whole model at 3 bits.

clamps). The squared sensitivity of the expert output to gate channel j at token t is $\sigma^{(1)}_{tj} = (\phi'(z_{tj})u_{tj})^2\|W_2[j,:]\|^2$, and to up channel j it is $\sigma^{(3)}_{tj} = \phi(z_{tj})^2\|W_2[j,:]\|^2$. We approximate each by the product of a column weight $s_j \propto \sum_t \sigma_{tj}$, normalized to mean 1, and a token weight $\omega_t = \sum_j \sigma_{tj}$, which gives $H = \sum_t \omega_t x_t x_t^\top / \sum_t \omega_t$. For the down projection, H comes from the quantized gate/up activations and $s = 1$. We shrink H 25% toward its diagonal and add 1% damping. The encoder runs reverse block-LDL error feedback with a weighted tail-biting Viterbi search over all 2^{16} states per 16-row block, in which each position's flag selects its value table. The path cost weights each position by the block-LDL diagonal and its column weight, $\delta_i s_j$.

MXFP4 operands. Because B consists of MXFP4 blocks, its product runs on existing block-scaled tensor-core GEMMs. A block-scaled FP4 or FP8 matrix multiply (W4A4, W4A8) takes the E2M1 codes and the block scales 2^{1-f} as they are and applies the column scale per output channel in the epilogue, and the tail correction is a sparse product added to the result. A W8A8 kernel sees the same values as exact FP8 numbers. The reference implementation of DeepSeek-V4.1-Flash quantizes each expert activation to E4M3 with one power-of-two scale per 32 inputs and multiplies it with the MXFP4 weights in a block-scaled FP4 GEMM. To run a GRAFT matrix on this path, a decoder first converts its trellis to MXFP4 codes and block scales. The GEMM itself is unchanged, and its result receives the column scale (one FP32 multiply per output) and the sparse tail product. With the native weights, the same path reproduces the reference bit for bit. In our evaluation the conversion of each layer's experts happens before the GEMM. This validates numerical compatibility, but decoding inside the GEMM, which keeps the weights compressed in memory, is not measured here.

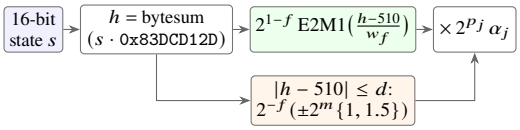


Figure 2: Decoding one weight. The state is the 16-bit window of the bitstream that ends at the current step, and the hash is EXL3's. GRAFT's value map is the hardware E2M1 conversion, scaled by the flag $f \in \{0, 1\}$ of the weight's 32-input block, or, for states at the hash centre, a power-of-two ladder (tail). No inverse rotation is needed. Outside the tail every block is an MXFP4 block with scale 2^{1-f}, and the tail values form an exact sparse correction.

Storage. Each matrix stores the trellis words, a rate code, a codebook code, six fp32 constants (q_a, q_b for each flag state and d), the fp16 column scales and, if the matrix uses flags, one bit per 32-input block and column. This amounts to $3kn + 16n + 208$ bits, plus $kn/32$ with flags, which gives 3.004 bits per weight on DeepSeek's experts without flags and 3.006 on average with them (EXL3 at 3.0 stores 3.010). The code consists of the trellis words alone, with no escapes or side tables.

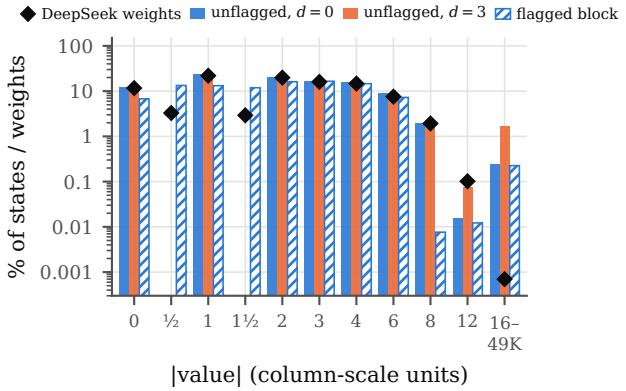


Figure 3: GRAFT's codebook compared with its source. Bars: share of the 65,536 trellis states that decode to each magnitude in an unflagged block (width 95.68) at tail band $d = 0$ and $d = 3$, and in a flagged block (hatched; its own table at width 53, the most common width in the fit, halved). The last bar pools the tail ladder $2^m\{1, 1.5\}$, $m = 3..15$, up to 49,152. Diamonds: share of DeepSeek-V4.1-Flash's routed-expert weights at each magnitude in the same column units (66 matrices). On the grid the codebook follows the source, including zero (12.2% of the states, 11.7% of the weights). The halves (6.2% of the weights) lie in blocks an octave below their column's MX scale, and a flagged block's table puts 25.2% of its states on them. To reach the outliers, which are rare (0.0007% of these weights lie above 12), the tail takes its states from the hash centre, where the bulk decodes to zero (10.6% of the states at $d = 3$). The fit uses $d = 0$ on 99.4% of the matrices.

5 Experiments

Setup. We code all 15,360 routed experts (46,080 matrices) of DeepSeek-V4.1-Flash, each at 3 bits per weight. Calibration uses up to 65,536 routed rows per expert (seed 41) plus every attention-sink row, drawn from a mixed web, code, math and multilingual corpus. Expert-level error is the NMSE of the expert output against the native MXFP4 expert on up to 1,024 held-out routed tokens per expert. We aggregate it over the model as the ratio of mean expert NMSE to EXL3's, with every expert weighted equally, and give the geometric mean of the per-expert ratios in parentheses. The baseline is EXL3 at 3.0 bits per weight as published (exllamav3's own quantizer, rotation and Hessian), fitted on the same calibration pool with the same seed and cap and measured on identical held-out rows. Model-level metrics use 400 held-out 2,048-token windows (all 140 of the WikiText-2 test set and 260 from C4 validation), scored against the native model with llama.cpp's rule, which scores the second half of each window without BOS (409,200 scored tokens). We report KL divergence, excess NLL of the true next token and top-1 agreement, with paired bootstrap 95% intervals over windows.

Damage and divergence. Our evaluation runs the experts in FP16 arithmetic. With the *native* weights, this path alone has KL 0.0189 to the reference (top-1 94.6%) at an excess NLL of 0.0002. Ulp-level differences swap near-tied top-k routes, and the swaps propagate through later layers [13, 15, 31], but the change in mean NLL on these corpora is negligible. Because KL also counts such route swaps, we take excess NLL as the measure of damage and KL as a measure of divergence, and we compare KL only between models in the same arithmetic. GRAFT's decoded weights also run on the reference's own expert kernels (Section 4). There the native weights reproduce the reference exactly, so every difference comes from GRAFT's weights.

Expert level. Over the whole model, GRAFT's mean held-out expert error is 0.630× that of EXL3 (geometric mean of the per-expert ratios 0.626×), and its error is lower on all but 11 of the 15,358 experts with held-out rows. The ratio is flat across depth (0.58–0.64 per layer; Figure 4a). This is what the discrete-source argument predicts for a source whose grid structure is the same in every layer. At 3 bits the memoryless reference is about 0.6× D_G, and GRAFT's tile error is about 1.3× that reference. On tiles (Figure 1b), where EXL3's codebook codes the same 66 DeepSeek matrices after EXL3's rotation, the median over matrices of GRAFT's error relative to EXL3's is 0.81× at 2.75 bits, 0.67× at 3, 0.45× at 3.25, 0.16× at 3.5 and 0.003× at 3.75. The advantage thus grows with the rate, and at 4 bits GRAFT reproduces every sampled tile of 91% of the matrices exactly.

Model level. GRAFT's excess NLL is 0.0085 nats against 0.0131 for EXL3, a ratio of 0.65× (paired difference −0.0045 [−0.0060, −0.0031]). Its KL is 0.0369 against 0.0469, a ratio of 0.79× (paired difference −0.0099 [−0.0106, −0.0093]), and its top-1 agreement is 92.5% against 91.7%. On the reference's own kernels, GRAFT's KL is 0.0384, slightly above that of the FP16 path (paired difference +0.0015 [0.0012, 0.0017]), and its excess NLL is 0.0095. Removing the arithmetic's own perturbation therefore does not lower GRAFT's KL. This is consistent with the route swaps caused by the arithmetic overlapping those caused by the weights rather than adding to them, although we did not trace individual routes to confirm the mechanism. Either way, KL measured in one arithmetic cannot be split into an arithmetic part and a weight part by subtraction.

Ablations. Table 2 quantifies GRAFT's design choices. Every row is fitted on the same calibration rows and measured on the same held-out rows. Where an ablation spends a different number of bits, we compare at equal bits using the measured held-out rate curve of the octave-codebook fit, which gives an error factor of 0.689× per extra quarter bit at 3 bits (14 experts).

Table 1: DeepSeek-V4.1-Flash at 3 bits per weight (routed experts). Expert: held-out expert NMSE relative to EXL3 on the same rows, ratio of means (geometric mean of ratios). Model level: 400 windows scored with the llama.cpp rule; excess NLL of the true next token (ΔNLL, nats), KL divergence and top-1 agreement against the native model, with the experts in FP16 arithmetic except on the reference kernels. The second row is a control that runs the native weights through the same FP16 arithmetic. Bits: stored bits per expert weight, all tensors included. †Ablation: per-matrix octave codebook instead of block flags (Table 2).

	bits	expert	ΔNLL	KL	top-1
native MXFP4	4.25	–	0	0	100%
FP16 expert arithmetic	4.25	–	0.0002	0.0189	94.6%
EXL3	3.010	1.00	0.0131	0.0469	91.7%
GRAFT	3.006	0.630 (0.626)	0.0085	0.0369	92.5%
reference kernels	3.006		0.0095	0.0384	92.4%
octave codebook†	3.004	0.631 (0.627)	0.0088	0.0371	92.4%

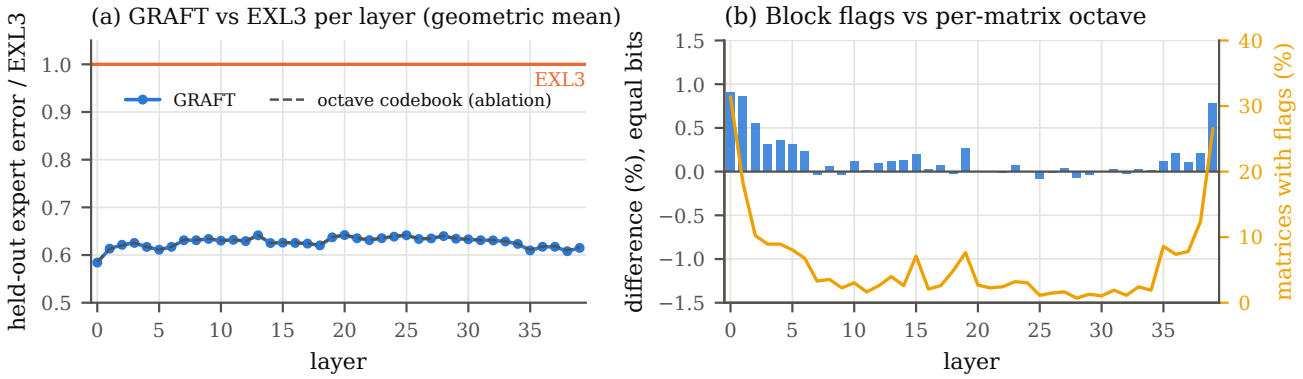


Figure 4: Held-out expert error per layer on DeepSeek-V4.1-Flash at 3 bits. (a) GRAFT and the octave-codebook ablation relative to EXL3 on the same rows (geometric means; the two curves overlap). (b) Difference between GRAFT's error and the octave codebook's at equal bits (bars), $100(\text{GRAFT}/\text{octave} - 1)$, where zero means equal error, and the share of matrices whose fit uses block flags (line).

- *The code or the fitting objective.* GRAFT's fit differs from EXL3's in the objective (gated, token-weighted H and column weights, shrinkage, down-projection statistics from the quantized gate/up activations) as well as in the code. An EXL3 refit with GRAFT's objective on the same calibration rows has 0.957× the held-out error of EXL3 as published (geometric mean 0.966×, 15,358 experts). GRAFT has 0.66× the error of this refit (geometric mean 0.65×) and is lower on all but 20 experts, so the objective accounts for a few percent and the grid-aligned code for the rest.
- *Block flags or a per-matrix octave codebook.* Instead of using flags, a matrix can move a fraction of its trellis states one octave down, so that states whose top bits are set decode at half value. The two approaches reach the same error. Over all 15,358 experts, the octave codebook has 1.001× GRAFT's held-out error at GRAFT's bits and 0.9985× at equal bits. The two are within ±0.3% in 33 of 40 layers and at most 0.9% apart in the first and last layers (Figure 4b). The model-level KL of the octave codebook is 0.0371, against 0.0369 for GRAFT (difference −0.0002 [−0.0004, 0.0001]). Flags are needed on 5.8% of the matrices and the octave on 18.9%, and only flags keep the decoded blocks MXFP4. On fit tiles at 3.5 bits the flags are ahead, with 0.78–0.81× the octave codebook's error on 48 matrices.
- *Flags off.* On the 14 flagged experts of the last layer, removing the flags raises held-out error by only 0.3% at equal bits, so at 3 bits the flags roughly break even on error. Their benefit is that they keep the decoded blocks MXFP4. They also decode in fewer instructions than the octave codebook (6.2 against 7.0 per weight on B200), and at 3.5 bits they are ahead (above).
- *Rate allocation.* Moving bits between matrices according to their predicted contribution to expert error, at the same stored size, lowers mean expert error by 27% (octave codebook, all experts) but KL by only 2%. It also lowers top-1 agreement (94.5% vs 94.9%) and raises excess NLL (0.0063 vs 0.0054 on 40 windows, too few to resolve the difference). Allocation by our expert-error estimator therefore did not improve model-level quality in this experiment, and GRAFT codes every matrix at the same rate.
- *The trellis or a 3-bit scalar code.* Scalar 3-bit codebooks of 8 values per 8 columns, fitted with the same objective and error feedback, have 2.89× GRAFT's held-out error at the same bits (16 experts in 4 layers). With plain

Table 2: Ablations: error relative to GRAFT (equal bits unless marked). Top: measured held-out expert error on the experts listed. Bottom: predictions of the selection estimator, not held-out measurements, given as the typical (geometric-mean) and total fit-weighted tile error of the octave-codebook fits over 36,192 matrices.

change	measured on	error
EXL3 code, GRAFT's objective	all experts	1.52×
octave codebook instead of flags	all experts	0.9985×
no block flags	14 experts, L39	1.003×
3-bit scalar tables, error feedback	16 experts	2.89×
plain rounding	16 experts	5.18×
+ MX scales (3.26 bits)	16 experts	2.61×
free (non-pow.-two) column scales	earlier code, 2 layers	+9%
tail band fixed at $d = 0$	estimator	+0.9% / +23%
no group multipliers	estimator	+0.0% / +2.9%

rounding they have 5.18×, and with an MX block scale per 32 weights at 3.26 bits they still have 2.61×. An 8-value table can hold at most 8 of the source's 15 levels, whereas the fitted trellis reproduces about 82% of the source weights exactly (within 5% of a grid step after the column scale, on octave-codebook fits of 20 experts at 3 bits).

6 Related work

Compressing low-precision releases. We know of no method that compresses an FP4-native checkpoint below 4 bits per weight by exploiting its grid. Tied Trit-Planes [14] requantizes DeepSeek-V4-Flash's MXFP4 experts to about 4 bits and treats the dequantized weights as continuous. ReQuant [26] refines codes on a fixed grid without lowering the bit width, and Zhao et al. [41] requantize FP8 DeepSeek releases to 3–4-bit GGUF types. Lossless compression gains almost nothing on MXFP4 and NVFP4 codes [16], which is consistent with our entropy measurements.

Trellis and vector quantization. GRAFT builds on QTIP's bitshift trellis with computed codebooks [36] as implemented in EXL3 [37], on LDLQ and incoherence processing [4, 35], and on GPTQ-style error feedback [11, 12]. Vector and additive codebooks [9, 25, 27, 38], Hadamard rotation with Gaussian-optimal grids [28] and fractional-bit allocation for Gaussianized weights [22] all assume a continuous source. Lattice VQ collapses without rotation on BF16 models [39], which is why a result without rotation is specific to sources already on a grid. Grid-aligned trellis quantizers exist for uniform integer grids, including classical TCQ [29], the dependent quantization of the NNR standard [20] and CCQ [42]. XOR-Trellis [32] quantizes BF16 checkpoints using a trellis that decodes to FP4 values with E5M3 group scales. Work on FP4 formats finds that rotations interact badly with block scales [10, 17, 34].

Objectives, outliers and allocation. GRAFT's gated objective is the special case of Kronecker-factored Hessians [2] with a per-expert, one-sided, diagonal output factor. Such a factor cancels in plain GPTQ [23], whereas GRAFT applies it through the trellis cost. Outliers are usually stored exactly beside the code [7, 18, 21, 24]. GRAFT instead gives the codebook itself a power-of-two range, which a grid-aligned code can afford because large MXFP4 weights are themselves powers of two times E2M1 values, and the stored code stays one bitstream. Sensitivity-driven mixed-precision allocation [8, 28] is orthogonal to the code. In our experiment on this model, moving bits between experts according to our expert-error estimates did not improve model-level quality (Section 5).

7 Limitations and conclusion

Limitations. We report results for one model; results on gpt-oss [30] are coming soon. We evaluate GRAFT at 3 bits per weight. Its tile-level advantage grows with the rate (Figure 1), but we do not evaluate higher rates end to end here. Model-level metrics use 400 windows from two text corpora, and we do not report capability benchmarks. We do not measure throughput or latency, and the reference-kernel evaluation decodes each layer's experts to MXFP4 operands before the matrix multiply.

Decode cost and speed. Like EXL3 and QTIP [36, 37], GRAFT decodes on the general-purpose cores rather than on the tensor cores. For each weight, the decoder extracts the 16-bit state (a shift and a mask), computes the hash (an integer multiply and a byte-sum dot product) and applies the value map (a fused multiply-add and half an FP4 conversion). On Blackwell (B200) this takes about 5.5 instructions in an unflagged block and 6.2 with a flag, against 7.0 for the per-matrix octave codebook. The smaller representation reduces weight traffic, but whether this decode work is hidden and the saving turns into throughput depends on overlapping the decode with memory access. Where decode itself sets the speed, as on HBM GPUs such as B200, a throughput model puts GRAFT's expert products at an estimated 2–3× the time of native MXFP4, so on these GPUs its benefit is capacity. The decode cost cannot be traded away cheaply. On 33 DeepSeek matrices (8 for the bit-sliced decoders), every cheaper state-to-value map we screened (top-bit and register tables, two weights per state, linear and bit-sliced decoders) raised weight error by 11–157%, whereas a random permutation of GRAFT's own table raised it by 3%. The byte-sum hash spreads the successors of a state better than random draws do. Hardware support for trellis decoding in the tensor-core operand path, like today's support for

2:4 sparsity metadata and MX block scales [33], would remove this cost for EXL3 and QTIP as well as for GRAFT.

Conclusion. An MXFP4 checkpoint is a discrete source, and a quantizer that keeps the grid instead of rotating it away can go well below the Gaussian-source curve for which rotation-based, Gaussian-matched coders are designed. GRAFT does this inside EXL3's trellis by changing only the value map. The new map applies an E2M1 conversion to EXL3's hash, with a one-bit MX scale per block and a power-of-two tail at the hash centre, so the bulk decodes to MXFP4 blocks again. On DeepSeek-V4.1-Flash, GRAFT has 0.63× EXL3's held-out expert error at 3 bits, and its MXFP4 blocks run on the model's own MXFP4 kernels with a per-column output scale and a sparse tail correction added. GRAFT therefore stores the experts in fewer than 4 bits per weight while the bulk of the computation stays in the format the model was trained in.

References

[1] Suguru Arimoto. An algorithm for computing the capacity of arbitrary discrete memoryless channels. *IEEE Transactions on Information Theory*, 18(1):14–20, 1972.

[2] Johann Birnick and Rayan Saab. FastKron: Efficient quantization with Kronecker-factored Hessians. arXiv:2608.06291, 2026.

[3] Richard E. Blahut. Computation of channel capacity and rate-distortion functions. *IEEE Transactions on Information Theory*, 18(4):460–473, 1972.

[4] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher De Sa. QuIP: 2-bit quantization of large language models with guarantees. In *Advances in Neural Information Processing Systems*, 2023.

[5] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley, 2nd edition, 2006.

[6] DeepSeek-AI, Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, et al. DeepSeek-V4: Towards highly efficient million-token context intelligence. arXiv:2606.19348, 2026.

[7] Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefler, and Dan Alistarh. SpQR: A sparse-quantized representation for near-lossless LLM weight compression. arXiv:2306.03078, 2023.

[8] Zhen Dong, Zhewei Yao, Amir Gholami, Michael Mahoney, and Kurt Keutzer. HAWQ: Hessian AWare quantization of neural networks with mixed-precision. In *ICCV*, 2019.

[9] Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. Extreme compression of large language models via additive quantization. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[10] Vage Egiazarian, Roberto L. Castro, Denis Kuznedelev, Andrei Panferov, Eldar Kurtic, Shubhra Pandit, et al. Bridging the gap between promise and performance for microscaling FP4 quantization. arXiv:2509.23202, 2025.

[11] Elias Frantar, Sidak Pal Singh, and Dan Alistarh. Optimal brain compression: A framework for accurate post-training quantization and pruning. In *Advances in Neural Information Processing Systems*, 2022.

[12] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023.

[13] Zhongqian Fu, Tianyi Zhao, Ning Ding, Xianzhi Yu, Xiaosong Li, Yehui Tang, and Yunhe Wang. EAQuant: Enhancing post-training quantization for MoE models via expert-aware optimization. arXiv:2506.13329, 2025.

[14] Matteo Grella. Tied trit-planes: Constraining PTQTP to a uniform nine-level quantizer, with a persistent folded format for disk-streamed mixture-of-experts serving. arXiv:2608.08910, 2026.

[15] Parvel Gu. Detecting a route flip is easier than knowing whether to fix it: Causal route-mediated damage in quantized mixture-of-experts. *arXiv preprint arXiv:2608.11212*, 2026.

[16] Anat Heilper and Doron Singer. Lossless compression of neural network components: Weights, checkpoints, and K/V caches in low-precision formats. arXiv:2508.19263, 2025.

[17] Ehsan Jokar. Transforms for LLM quantization: The great inversion and format co-design. arXiv:2608.25188, 2026.

[18] Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W. Mahoney, and Kurt Keutzer. SqueezeLLM: Dense-and-sparse quantization. In *International Conference on Machine Learning*, 2024.

[19] Kimi Team. Kimi K3: Open frontier intelligence, 2026. MXFP4 expert weights; quantization-aware training throughout post-training (SFT and RL).

[20] Heiner Kirchhoffer, Paul Haase, Wojciech Samek, Karsten Müller, Hamed Rezazadegan-Tavakoli, Francesco Cricri, et al. Overview of the neural network compression and representation (NNR) standard. *IEEE Transactions on Circuits and Systems for Video Technology*, 32(5):3203–3216, 2022. doi: 10.1109/TCSVT.2021.3095970.

[21] Changhun Lee, Jungyu Jin, Taesu Kim, Hyungjun Kim, and Eunhyeok Park. OWQ: Outlier-aware weight quantization for efficient fine-tuning and inference of large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2024.

[22] Deokjae Lee and Hyun Oh Song. Q-Palette: Fractional-bit quantizers toward optimal bit allocation for efficient LLM deployment. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[23] Donghyun Lee, Yuhang Li, Ruokai Yin, and Priyadarshini Panda. KronQ: LLM quantization via Kronecker-factored Hessian. In *Conference on Language Modeling (COLM)*, 2026.

[24] Xinlin Li, Osama Hanna, Christina Fragouli, and Suhas Diggavi. ICQuant: Index coding enables low-bit LLM quantization. arXiv:2505.00850, 2025.

[25] Yifei Liu, Jicheng Wen, Yang Wang, Shengyu Ye, Li Lyna Zhang, Ting Cao, Cheng Li, and Mao Yang. VPTQ: Extreme low-bit vector post-training quantization for large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

[26] Yongge Ma, Guoan Wang, Feiyu Wang, Yaoming Li, Qian Zhang, Zihan Yan, Yinjun Han, and Tong Yang. ReQuant: Fixed-grid discrete refinement for post-training quantization. arXiv:2608.07019, 2026.

[27] Vladimir Malinovskii, Denis Mazur, Ivan Ilin, Denis Kuznedelev, Konstantin Burlachenko, Kai Yi, Dan Alistarh, and Peter Richtárik. PV-Tuning: Beyond straight-through estimation for extreme LLM compression. arXiv:2405.14852, 2024.

[28] Vladimir Malinovskii, Andrei Panferov, Ivan Ilin, Han Guo, Peter Richtárik, and Dan Alistarh. HIGGS: Pushing the limits of large language model quantization via the linearity theorem. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 10857–10886. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.naacl-long.543.

[29] Michael W. Marcellin and Thomas R. Fischer. Trellis coded quantization of memoryless and Gauss-Markov sources. *IEEE Transactions on Communications*, 38 (1):82–93, 1990. doi: 10.1109/26.46532.

[30] OpenAI, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, et al. gpt-oss-120b & gpt-oss-20b model card. arXiv:2508.10925, 2025.

[31] Hancheol Park, Geonho Lee, Tairen Piao, and Tae-Ho Kim. Value-and-structure alignment for routing-consistent quantization of mixture-of-experts models. *arXiv preprint arXiv:2606.05688*, 2026.

[32] Xiaofan Que, Nir Elkayam, Spandan Pyakurel, Shuokai Pan, and Dibakar Gope. XOR-Trellis: Ultra-low-complexity dequantization and curvature-aware Hadamard-free LLM quantization. arXiv:2610.00432, 2026.

[33] Bita Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, et al. Microscaling data formats for deep learning. arXiv:2310.10537, 2023.

[34] Yuantian Shao, Peisong Wang, Yuanteng Chen, Chang Xu, Zhihui Wei, and Jian Cheng. Block rotation is all you need for MXFP4 quantization. arXiv:2511.04214, 2025.

[35] Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks. In *International Conference on Machine Learning*, 2024.

[36] Albert Tseng, Qingyao Sun, David Hou, and Christopher De Sa. QTIP: Quantization with trellises and incoherence processing. In *Advances in Neural Information Processing Systems*, 2024.

[37] turboderp. ExLlamaV3. https://github.com/turboderp-org/exllamav3, 2025. Software, version 1.5.3.

[38] Mart van Baalen, Andrey Kuzmin, Ivan Koryakovskiy, Markus Nagel, Peter Couperus, Cedric Bastoul, et al. GPTVQ: The blessing of dimensionality for LLM quantization. arXiv:2402.15319, 2024.

[39] Tycho F. A. van der Ouderaa, Mart van Baalen, Paul Whatmough, and Markus Nagel. Leech lattice vector quantization for efficient LLM compression. arXiv:2603.11021, 2026.

[40] Xiaomi MiMo. MiMo-V2.5-Pro-FP4-DFlash model card. https://huggingface.co/XiaomiMiMo/MiMo-V2.5-Pro-FP4-DFlash, 2026. MXFP4 MoE experts, FP4 QAT.

[41] Enbo Zhao, Yi Shen, Shuming Shi, Jieyun Huang, Zhihao Chen, Ning Wang, et al. Quantitative analysis of performance drop in DeepSeek model quantization. arXiv:2505.02390, 2025.

[42] Zhaojing Zhou, Xunchao Li, Minghao Li, Handi Zhang, Haoshuang Wang, Wenbin Chang, et al. CCQ: Convolutional code for extreme low-bit quantization in LLMs. arXiv:2507.07145, 2025.